

Findings - Task 2 Level 1

Why residual connections help optimization

In very deep networks, oftentimes there is a degradation problem, as the randomly initialised weights in the beginning can get scrambled up and lose context for larger depths, which prevents the gradients from meaningfully updating the weights. Similarly during backpropagation, the flow of weights from the end of the network might cause the loss of original meaning in the earlier layers. Residual connections prevent this by changing what every block needs to learn. Instead of having to compute the entire transformation, the blocks only need to find out the “residual”, which is an easier and less computationally expensive task.

How skip connections improve gradient flow

Skip connections provide a weight-free path for gradients to flow through so that the initial context is maintained without getting manipulated by any weights. This is how meaning is preserved throughout multiple layers with any vanishing/exploding gradient problems.

The effect of network depth on performance

Three network architectures were tested, at varying capacities. The first one was a 3-layer ResNet with the first having 2 blocks and the rest having 3 each. The depth of features increased from 32 to 64 to 128 to 256, with the classifier having an extra 256-512 layer. This network showed a validation accuracy of 91%. The second one was a 4-layer network modelled as 2,3,3,3. The increased depth was accompanied by the same width from 32 to 256 along with the same classifier layer. This saw an accuracy of 90.6%, proving that increasing the depth doesn't necessarily improve accuracy if suitable regularisation and dropout are not provided (the weight decay and dropout were kept the same), and that the feature width affected parameter count far more than network depth. The final network was also of 4 layers, but the feature depth was increased from 64 to 128 to 256 to 512, with no hidden classifier layer to prevent further overfitting. The weight decay was also increased to $5e^{-4}$ to handle the increase in network capacity and prevent overfitting. This gave similar results only, with no stark difference from the other accuracies. Thus, it can be said that the initial 2,3,3 model was successful enough in extracting the important features from the dataset, and the layer added after that even with increased feature depth did not show much change and acted as wasted capacity, meaning that capacity saturation had taken space. Oftentimes, datasets like CIFAR-10 do not require very deep networks with high width, and the depth can cause diminishing results. The 2,3,3 model was chosen for this reason and also because it used ~10M less parameters than the last model, inspired from the ResNet-18 architecture.

```

Best validation accuracy: 90.60%
Best model weights saved: model_weights.pkl
Plots saved successfully: training_curves_cnn.png
Confusion matrix created successfully: confusion_matrix_cnn.png

```

	precision	recall	f1-score	support
airplane	0.89	0.93	0.91	1000
automobile	0.96	0.96	0.96	1000
bird	0.88	0.86	0.87	1000
cat	0.84	0.76	0.80	1000
deer	0.89	0.91	0.90	1000
dog	0.84	0.86	0.85	1000
frog	0.91	0.94	0.93	1000
horse	0.95	0.93	0.94	1000
ship	0.94	0.96	0.95	1000
truck	0.95	0.95	0.95	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

```

Validation Loss: 0.723710487651825 | Validation Accuracy: 90.60000000000001%

```

2,3,3,3, model with no change in regularisation or dropout

```

Epoch: 1 | Training Loss: 2.1514170266723633 | Training Accuracy:
27.004% | Validation Loss: 1.6611635553359985 | Validation Accuracy:
45.08%

```

```

Epoch: 5 | Training Loss: 1.4654292900085448 | Training
Accuracy: 56.53399999999999% | Validation Loss: 1.2819510385513306 |
Validation Accuracy: 64.270000000000001%

```

```

Epoch: 10 | Training Loss: 1.2410903038787842 | Training
Accuracy: 67.328% | Validation Loss: 1.0425186451911925 | Validation
Accuracy: 75.82%

```

```

Epoch: 15 | Training Loss: 1.1648112743377685 | Training
Accuracy: 71.07% | Validation Loss: 0.9736654231071472 | Validation
Accuracy: 79.54%

```

```

Epoch: 20 | Training Loss: 1.12571039894104 | Training
Accuracy: 72.938% | Validation Loss: 0.9185865343093872 | Validation
Accuracy: 81.89999999999999%

```

```

Epoch: 25 | Training Loss: 1.0956108445739745 | Training
Accuracy: 74.5% | Validation Loss: 0.9361758255958557 | Validation
Accuracy: 81.47%

```

```

Epoch: 30 | Training Loss: 1.0774459882164003 | Training
Accuracy: 75.334% | Validation Loss: 0.8955428840637207 | Validation
Accuracy: 83.19%

```

```
Epoch: 35 | Training Loss: 1.048906524734497 | Training  
Accuracy: 76.292% | Validation Loss: 0.8600618730545044 | Validation  
Accuracy: 84.71%  
Epoch: 40 | Training Loss: 1.023843419265747 | Training  
Accuracy: 77.602% | Validation Loss: 0.8497398956298828 | Validation  
Accuracy: 84.82%  
Epoch: 45 | Training Loss: 1.0036889081192018 | Training  
Accuracy: 78.44% | Validation Loss: 0.8365554956436158 | Validation  
Accuracy: 85.75%  
Epoch: 50 | Training Loss: 0.9725225496673584 | Training  
Accuracy: 79.89% | Validation Loss: 0.8081493230819702 | Validation  
Accuracy: 86.81%  
Epoch: 55 | Training Loss: 0.9466869074249268 | Training  
Accuracy: 81.126% | Validation Loss: 0.7990646743774414 | Validation  
Accuracy: 87.58%
```

2,3,3,3 with no classifier layer and increased weight decay

Challenges encountered during implementation

1. **Architectural consistency:** Ensuring each residual block correctly matched the ResNet pattern (conv - BN - Relu - add residual - conv - BN), an early version did ReLU before the skip connection adding which deviates from the standard block pattern.
2. **Channel/dimension matching for shortcuts:** Projection shortcuts (1×1 conv + BatchNorm) were necessary whenever channel counts changed between stages since the residual addition requires both branches to have identical shapes.
3. **Choosing the right architecture:** A lot of experimentation was done with the architecture and choosing the right depth and width while keeping the parameter count and computational cost in mind. Inspiration was taken from the ResNet-18 model.